SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Fundamentals of Data Science **COURSE CODE: DSA04**

COURSE OUTCOME COVERED

CO2: Apply data pre-processing and visualization techniques using Python libraries for effective data handling. (BL4)

QUESTIONS: 05 **Total Marks:100**

Annexure A

TECHNICAL PRESENTATION

Code of Conduct:

I N Abishek Kumaran /(192573006) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

RUBRICS FOR CALCULATION

TECHNICAL PRESENTATION					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Annexure A

Technical Presentation

5. Smart Agriculture Data Monitoring

An agriculture research team collects temperature, humidity, soil moisture, and crop growth data from sensors. Explain how:

A. NumPy is used for numerical computation
B. Pandas is used for organizing sensor data
C. Scikit-Learn and visualization libraries support crop condition analysis

A. NumPy for Numerical Computation

NumPy is a Python library used to perform numerical calculations efficiently. In smart agriculture, it can calculate the mean, minimum, and maximum values of temperature readings collected from sensors.

Code and Output:

```
import numpy as np

temperature = np.array([28, 30, 31, 29, 32])

print("Temperature readings:", temperature)
print("Mean temperature:", np.mean(temperature))
print("Minimum temperature:", np.min(temperature))
print("Maximum temperature:", np.max(temperature))
```

```
Temperature readings: [28 30 31 29 32]
Mean temperature: 30.0
Minimum temperature: 28
Maximum temperature: 32
```

Result: The mean temperature is 30°C, the minimum temperature is 28°C, and the maximum temperature is 32°C.

B. Pandas for Organizing Sensor Data

Pandas is used to organize sensor readings into a DataFrame. It helps identify missing values, clean incomplete data, and generate summary statistics.

Code and Output:

1. The original sensor-data table and missing-value count.

```
data = {
    "Day": [1, 2, 3, 4, 5],
    "Temperature": [28, 30, np.nan, 31, 32],
    "Humidity": [65, 62, 60, 59, 58],
    "Soil_Moisture": [32, 30, 28, 27, 25]
}

df = pd.DataFrame(data)

print("Original sensor data:")
display(df)

print("Missing values:")
print(df.isnull().sum())
```

```
Original sensor data:
```

	Day	Temperature	Humidity	Soil_Moisture
0	1	28.0	65	32
1	2	30.0	62	30
2	3	NaN	60	28
3	4	31.0	59	27
4	5	32.0	58	25

```
Missing values:
Day             0
Temperature     1
Humidity        0
Soil_Moisture   0
dtype: int64
```

2. The cleaned sensor-data table.

```
df["Temperature"] = df["Temperature"].fillna(
    df["Temperature"].median()
)

print("Cleaned sensor data:")
display(df)

print("Missing values after cleaning:")
print(df.isnull().sum())
```

```
Cleaned sensor data:
```

	Day	Temperature	Humidity	Soil_Moisture
0	1	28.0	65	32
1	2	30.0	62	30
2	3	30.5	60	28
3	4	31.0	59	27
4	5	32.0	58	25

```
Missing values after cleaning:
Day              0
Temperature      0
Humidity         0
Soil_Moisture    0
dtype: int64
```

3. The summary statistics output.

```
print("Summary Statistics:")
display(df.describe().round(2))
```

```
Summary Statistics:
```

	Day	Temperature	Humidity	Soil_Moisture
count	5.00	5.00	5.00	5.0
mean	3.00	30.30	60.80	28.4
std	1.58	1.48	2.77	2.7
min	1.00	28.00	58.00	25.0
25%	2.00	30.00	59.00	27.0
50%	3.00	30.50	60.00	28.0
75%	4.00	31.00	62.00	30.0
max	5.00	32.00	65.00	32.0

Result: Pandas identifies the missing temperature value and replaces it with the median of the available temperature readings.

C. Scikit-Learn and Visualization Libraries

Scikit-Learn provides machine-learning algorithms that can be used to demonstrate crop-condition classification. Matplotlib and Seaborn help visualize sensor readings, identify trends, and explore relationships between agricultural parameters.

Code and Output:

1. The Scikit-Learn code, predictions, and accuracy.

```
X = model_data[
    ["Temperature", "Humidity", "Soil_Moisture"]
]

y = model_data["Condition"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

model = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("Actual conditions:", list(y_test))
print("Predicted conditions:", list(predictions))

print(
    "Model accuracy:",
    round(accuracy_score(y_test, predictions) * 100, 2),
    "%"
)
```

```
Actual conditions: ['Needs Attention', 'Normal', 'Normal', 'Normal', 'Normal']
Predicted conditions: ['Needs Attention', 'Normal', 'Normal', 'Normal', 'Normal']
Model accuracy: 100.0 %
```

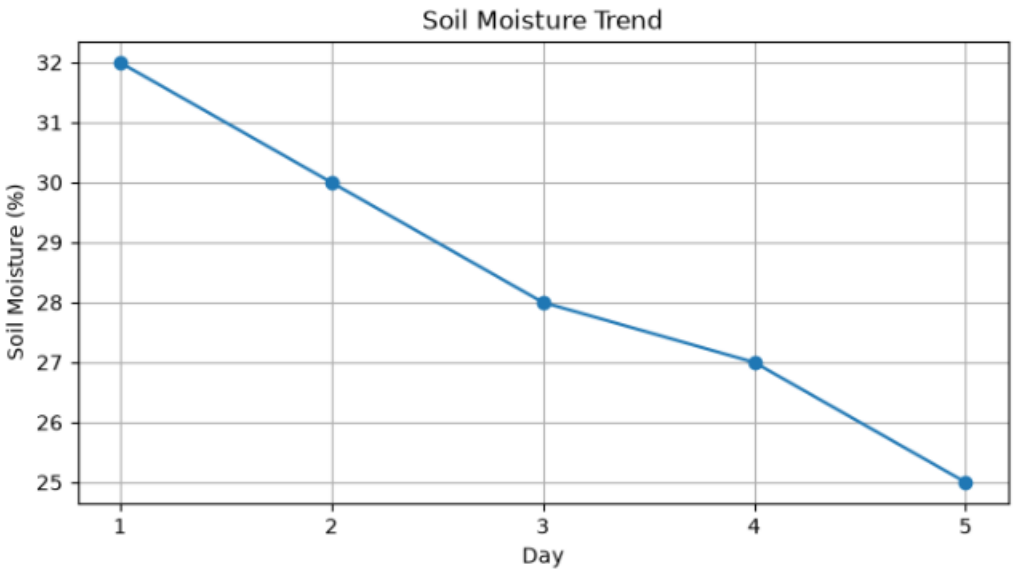
2. The soil-moisture line graph.

```
plt.figure(figsize=(8, 4))

plt.plot(
    df["Day"],
    df["Soil_Moisture"],
    marker="o"
)

plt.title("Soil Moisture Trend")
plt.xlabel("Day")
plt.ylabel("Soil Moisture (%)")
plt.xticks(df["Day"])
plt.grid(True)

plt.show()
```



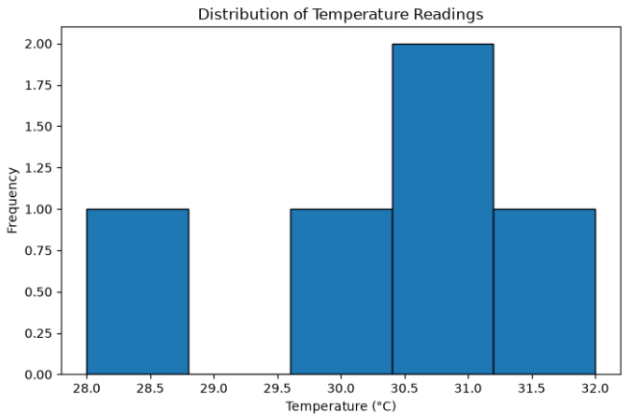
3. The temperature-distribution histogram.

```
plt.figure(figsize=(8, 5))

plt.hist(
    df["Temperature"],
    bins=5,
    edgecolor="black"
)

plt.title("Distribution of Temperature Readings")
plt.xlabel("Temperature (°C)")
plt.ylabel("Frequency")

plt.show()
```



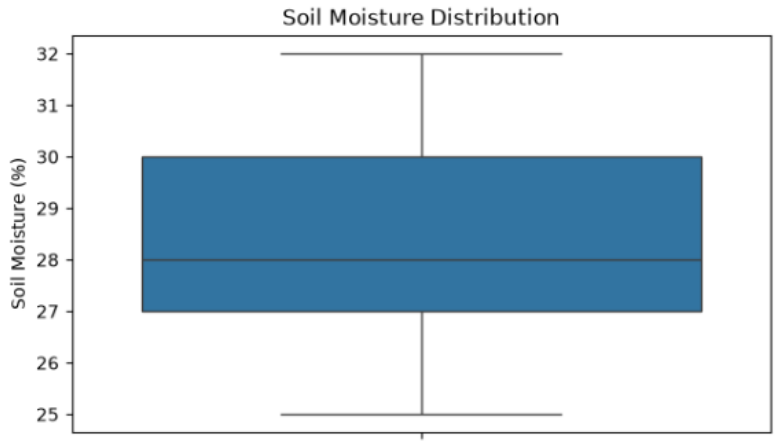
4. The soil-moisture box plot.

```
plt.figure(figsize=(7, 4))

sns.boxplot(y=df["Soil_Moisture"])

plt.title("Soil Moisture Distribution")
plt.ylabel("Soil Moisture (%)")

plt.show()
```



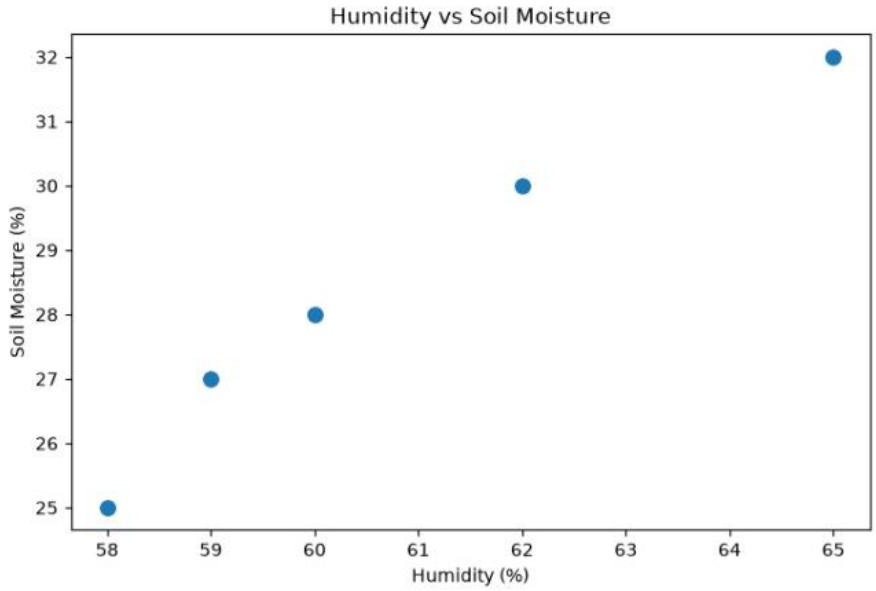
5. The humidity-versus-soil-moisture scatter plot.

```
plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="Humidity",
    y="Soil_Moisture",
    s=100
)

plt.title("Humidity vs Soil Moisture")
plt.xlabel("Humidity (%)")
plt.ylabel("Soil Moisture (%)")

plt.show()
```



Result: The model demonstrates classification using sample data, while the graphs help visualize sensor measurements and their relationships.

Observations

1. The average temperature of the sample readings is 30°C.
2. Soil moisture decreases from 32% on Day 1 to 25% on Day 5.
3. Pandas detects and handles the missing temperature value.
4. Visualizations help identify trends and relationships in sensor data.
5. Scikit-Learn demonstrates the process of training and testing a classification model.

Conclusion

This assessment demonstrates the use of NumPy, Pandas, Scikit-Learn, Matplotlib, and Seaborn for smart agriculture data monitoring. These libraries support numerical computation, sensor-data processing, example crop-condition classification, and visualization. The dataset and crop-condition labels used in this demonstration are illustrative and are not validated for real agricultural decisions.